

Donatón

Integrantes

Gonzalo Aliaga

Matias Cristi

Nicolas Rojas

Docente

Claudio Esteban Rojas Rodriguez

1. Contexto y Problemática

Donatón es una plataforma diseñada para coordinar la recepción, almacenamiento y distribución de donaciones hacia comunidades afectadas por emergencias. El sistema conecta cuatro actores principales: administradores, donantes, personal de logística y voluntarios, cada uno con necesidades funcionales distintas que exigen una separación clara de responsabilidades.

El problema central que resuelve la plataforma es la falta de trazabilidad entre la donación recibida, el inventario disponible en los centros de acopio y la entrega efectiva del recurso a quien lo necesita. Esto se traduce en tres dominios de negocio independientes:

- **Donaciones:** registro del recurso donado, su origen, cantidad y el centro de acopio al que ingresa.
- **Inventario:** administración de los centros de acopio (las "casas" físicas) y el stock de cada recurso por centro.
- **Logística:** necesidades reportadas en terreno, transporte disponible y envíos planificados hasta la entrega final.

Dada la heterogeneidad de estos dominios y la necesidad de escalarlos de forma independiente, se optó por una arquitectura de microservicios en vez de un monolito, exponiendo un único punto de entrada al frontend mediante un patrón Backend For Frontend.

2. Arquitectura del Sistema

2.1 Nivel C1

[Enlace a diagrama C1](#)

2.2 Nivel C2

[Enlace a diagrama C2](#)

2.3 Nivel C3

[Diagrama C3 Auth](#)

[Diagrama C3 Profile](#)

[Diagrama C3 Donation](#)

[Diagrama C3 Inventory](#)

[Diagrama C3 Logistic](#)

3. Patrones de Diseño Aplicados

3.1 Backend For Frontend (BFF)

El servicio bff actúa como punto de entrada único para el frontend, ocultándole la topología interna de microservicios. Expone endpoints agregados (/api/auth, /api/profile, /api/donations, /api/collection-centers, /api/inventory-items, /api/needs, /api/transport, /api/shipments) que internamente reenvía al microservicio correspondiente, agregando el header de autorización.

3.2 Gateway / Proxy

Dentro del BFF, cada microservicio downstream tiene un Client dedicado (AuthClient, ProfileClient, DonationClient, InventoryClient, LogisticClient) que encapsula la comunicación HTTP saliente mediante RestTemplate, centralizando el manejo de errores de red y de los códigos de estado HTTP devueltos por cada servicio.

3.3 Repository Pattern

Cada microservicio usa Spring Data JPA mediante interfaces JpaRepository (por ejemplo, DonationRepository, InventoryItemRepository, ShipmentRepository), desacoplando la lógica de negocio del mecanismo concreto de persistencia.

3.4 Factory Pattern

AuthResponseFactory en ms-auth centraliza la construcción de la respuesta de autenticación exitosa, evitando duplicar la lógica de ensamblado del DTO de respuesta en distintos puntos del servicio.

3.5 Centralized Exception Handling

Todos los microservicios, incluido el BFF, implementan un `GlobalExceptionHandler` anotado con `@RestControllerAdvice` que intercepta excepciones de negocio (`NotFoundException`, `BadRequestException`, `InsufficientStockException`, `TransportUnavailableException`, etc.) y errores de validación (`MethodArgumentNotValidException`), traduciéndolos a respuestas HTTP consistentes con un cuerpo `ErrorResponse` uniforme (status, error, message, timestamp). Esto evita manejo de errores disperso en cada controlador y garantiza una experiencia de API predecible.

3.6 DTO Pattern

La comunicación externa de cada microservicio nunca expone directamente las entidades JPA; se usan DTOs específicos por operación (`CreateDonationDto`, `UpdateDonationStatusDto`, `CreateShipmentDto`, `AssignTransportDto`, etc.), reduciendo el acoplamiento entre el modelo de persistencia y el contrato público de la API.

3.7 Database per Service

Cada microservicio de dominio posee su propia base de datos PostgreSQL aislada (`auth_db`, `profile_db`, `donations_db`, `inventory_db`, `logistic_db`). No existen joins ni foreign keys entre bases de datos distintas; las referencias cruzadas (por ejemplo, `Shipment.donationId` o `Shipment.originCenterId`) se almacenan como IDs simples, replicando el enfoque que usan sistemas distribuidos reales para evitar acoplamiento a nivel de almacenamiento.

4. Modelo de Datos y Persistencia

La persistencia se implementa con Spring Data JPA sobre PostgreSQL en cada microservicio, y las migraciones de esquema se gestionan con Flyway, garantizando que el esquema de cada base de datos sea versionado y reproducible en cualquier entorno (local, CI o Kubernetes).

4.1 ms-auth (auth_db)

Tabla `users` (id, username, password). El servicio `JwtService` firma tokens JWT con algoritmo RS256 usando un par de claves RSA (privada/pública) inyectadas vía variables de entorno.

4.2 ms-profile (profile_db)

Tabla `profiles` (id, role, email, address, run). El id del perfil corresponde al mismo id emitido por ms-auth, permitiendo al BFF encadenar `GET /api/profile/{id}` tras un login exitoso para obtener el rol real del usuario.

4.3 ms-donation (donations_db)

Tabla `donations` (id, resource, quantity, origin, donation_date, collection_center_id, status). Se sigue el ciclo: PENDING - RECEIVED - ASSIGNED - DELIVERED.

4.4 ms-inventory (inventory_db)

Tablas `collection_centers` (id, name, address, region, capacity, active) e `inventory_items` (id, collection_center_id, resource, quantity, unit, last_updated), con FK real entre ambas dado que pertenecen al mismo bounded context.

4.5 ms-logistic (logistic_db)

Tres tablas relacionadas dentro del mismo servicio: `needs` (necesidades reportadas en terreno), `transports` (vehículos y conductores disponibles) y `shipments` (envíos, con FK opcional hacia `need_id` y `transport_id`). `ShipmentService` orquesta la reserva y liberación automática de transporte según el estado del envío.

5. Decisiones Críticas de Diseño

5.1 Autenticación centralizada en el BFF

Se decidió que solo el BFF valide el token JWT mediante `JwtValidationFilter` antes de reenviar la petición. Los microservicios de dominio (donation, inventory, logistic) no validan el JWT por sí mismos en esta iteración, confiando en que toda petición proviene del BFF. Esta decisión simplifica la implementación pero introduce un riesgo: si un microservicio quedara expuesto directamente fuera del cluster, no tendría protección propia.

5.2 Separación de ms-auth y ms-profile

Se optó por separar la identidad (credenciales, token) del perfil (rol, datos de contacto) en dos microservicios distintos en vez de uno solo. Esto permite que el ciclo de vida de seguridad (rotación de contraseñas, tokens) evolucione independientemente de los datos de perfil, y refleja una separación de responsabilidades estándar en sistemas de autenticación modernos.

5.3 ms-logistic como microservicio unificado

Inicialmente se evaluó separar Necesidades, Transporte y Envíos en microservicios independientes. Se decidió unificarlos en ms-logistic porque las tres entidades comparten el mismo ciclo de vida transaccional: un envío siempre coordina un transporte y, opcionalmente, cubre una necesidad. Separarlos habría requerido transacciones distribuidas o un patrón para mantener consistencia.

5.4 H2 en memoria para pruebas unitarias

Para que los tests de contexto de Spring Boot (`@SpringBootTest`) no dependan de una instancia PostgreSQL real, cada microservicio usa una base de datos H2 en memoria activada por el perfil `test`, con Flyway deshabilitado. Esto permite ejecutar la suite completa de pruebas en cualquier entorno sin infraestructura adicional.

5.5 IDs simples

Se decidió que ms-donation y ms-logistic referencien centros de acopio mediante un `Long` simple, en vez de realizar una llamada HTTP síncrona a ms-inventory para validar su existencia en cada escritura. Esto evita acoplamiento temporal entre servicios (si ms-inventory cae, ms-donation sigue funcionando) a costa de no garantizar consistencia inmediata; se asume que la validación de existencia del centro ocurre en la capa de presentación (frontend) al momento de seleccionar el centro desde un listado ya cargado.

6. Pruebas Unitarias y Cobertura

Todos los microservicios usan JUnit 5 como runner y MockK como framework de mocking. Se priorizó cobertura en las capas Service y Controller (donde reside la lógica de negocio) y en el GlobalExceptionHandler de cada servicio.

Microservicio	Clases bajo prueba	Casos cubiertos
ms-auth	AuthService, AuthController, JwtService, GlobalExceptionHandler	Login exitoso/fallido, actualización de username, generación/validación de JWT
ms-profile	ProfileService, ProfileController, GlobalExceptionHandler	Obtención y actualización parcial de perfil, fusión de campos opcionales
ms-donation	DonationService, DonationController, GlobalExceptionHandler	Ciclo de vida de estados, filtros por centro y estado
ms-inventory	CollectionCenterService, InventoryItemService, ambos Controllers	Ajuste de stock positivo/negativo, validación de stock insuficiente
ms-logistic	NeedService, TransportService, ShipmentService, 3 Controllers	Reserva/liberación de transporte, transición de estados de envío
bff	BffAuthService, BffProfileService, BffDonationService, BffInventoryService, BffLogisticService	Delegación correcta a cada Client con sus parámetros

7. Integración Frontend — Backend

El frontend (React 19 + TypeScript + Vite) se comunica exclusivamente con el BFF en `http://localhost:8080` mediante un cliente HTTP centralizado (`apiRequest`) que adjunta el header `Authorization: Bearer <token>` en cada petición autenticada.

El flujo de inicio de sesión implementado es el siguiente:

1. El usuario ingresa username y password en el formulario de Login.
2. El frontend hace `POST /api/auth/login` en el BFF, que lo reenvía a `ms-auth`.
3. `ms-auth` valida las credenciales y retorna `{id, username, token}` firmado con la clave privada RSA.
4. El frontend encadena automáticamente `GET /api/profile/{id}` usando el token recién obtenido.
5. El BFF reenvía la petición a `ms-profile`, que retorna el rol y los datos de perfil del usuario.
6. El frontend ensambla un objeto `User` completo (identidad + rol) y lo persiste en el `AuthContext`, redirigiendo al dashboard correspondiente según el rol.

Este diseño evita que el JWT cargue información de rol directamente en su payload, manteniendo el rol como un dato mutable gestionado por `ms-profile` en vez de un claim estático del token.

8. Infraestructura y Despliegue

Cada microservicio incluye su propio Dockerfile multi-etapa (build con Gradle, ejecución con JRE Alpine) y manifiestos de Kubernetes independientes en su carpeta k8s/, compuestos por un Deployment + Service para la aplicación y otro para su base de datos PostgreSQL dedicada, con un initContainer que espera a que la base de datos esté lista antes de iniciar el microservicio.

Para desarrollo local se usa Docker Compose, levantando los 6 servicios de aplicación junto a sus 5 bases de datos PostgreSQL en la misma red de contenedores.

9. Conclusiones

La arquitectura implementada cumple con los principios fundamentales de microservicios: cada servicio posee una única responsabilidad de negocio, su propia base de datos, y se comunica exclusivamente mediante API REST documentada con Swagger/OpenAPI. El patrón BFF centraliza la seguridad y simplifica el consumo desde el frontend, mientras que el manejo de errores centralizado en cada servicio garantiza respuestas consistentes ante fallos.